

Android Reverse Engineering & API Extraction – Claude Code skill

License Apache-2.0

Stars 6.3k

last commit june

A Claude Code skill that decompiles Android APK/XAPK/JAR/AAR files and **extracts the HTTP APIs** used by the app – Retrofit endpoints, OkHttp calls, hardcoded URLs, authentication patterns – so you can document and reproduce them without the original source code.

Windows / PowerShell support (experimental): The *.ps1 scripts alongside the bash ones are a recent community contribution, still being stabilised. For any issues please open an issue on **this** repository (not on the contributors’ upstream forks): the PowerShell scripts are maintained here by [@SimoneAvogadro](#).

Table of Contents

- [What it does](#)
- [Requirements](#)
- [Installation](#)
- [Usage](#)
- [Repository Structure](#)
- [References](#)
- [Acknowledgments](#)
- [Disclaimer](#)
- [License](#)

What it does

Capability	Description
Decompile	APK, XAPK, JAR, and AAR files using jadx and Fernflower/Vineflower (single engine or side-by-side comparison)
Extract APIs	Retrofit endpoints, OkHttp calls, hardcoded URLs, auth headers and tokens
Trace call flows	From Activities/Fragments through ViewModels and repositories down to HTTP calls
Analyze structure	Manifest, packages, architecture patterns
Handle obfuscation	Strategies for navigating ProGuard/R8 output

Requirements

Required:

- Java JDK 17+
- [jadx](#) (CLI)

Optional (recommended):

- [Vineflower](#) or [Fernflower](#) – better output on complex Java code
- [dex2jar](#) – needed to use Fernflower on APK/DEX files

See `plugins/android-reverse-engineering/skills/android-reverse-engineering/references/setup-guide.md` for detailed installation instructions.

Installation

From GitHub (recommended)

Inside Claude Code, run:

```
/plugin marketplace add SimoneAvogadro/android-reverse-engineering-skill  
/plugin install android-reverse-engineering@android-reverse-engineering-skill
```

The skill will be permanently available in all future sessions.

From a local clone

```
git clone https://github.com/SimoneAvogadro/android-reverse-engineering-skill.git
```

Then in Claude Code:

```
/plugin marketplace add /path/to/android-reverse-engineering-skill  
/plugin install android-reverse-engineering@android-reverse-engineering-skill
```

Usage

Slash command

```
/decompile path/to/app.apk
```

This runs the full workflow: dependency check, decompilation, and initial structure analysis.

Natural language

The skill activates on phrases like:

- “Decompile this APK”
- “Reverse engineer this Android app”
- “Extract API endpoints from this app”
- “Follow the call flow from LoginActivity”
- “Analyze this AAR library”

Manual scripts

The scripts can also be used standalone:

```

# Check dependencies
bash plugins/android-reverse-engineering/skills/android-reverse-engineering/scripts/check-deps.sh

# Install a missing dependency (auto-detects OS and package manager)
bash plugins/android-reverse-engineering/skills/android-reverse-engineering/scripts/install-dep.sh jadx
bash plugins/android-reverse-engineering/skills/android-reverse-engineering/scripts/install-dep.sh vineflower

# Decompile APK with jadx (default)
bash plugins/android-reverse-engineering/skills/android-reverse-engineering/scripts/decompile.sh app.apk

# Decompile XAPK (auto-extracts and decompiles each APK inside)
bash plugins/android-reverse-engineering/skills/android-reverse-engineering/scripts/decompile.sh app-bundle.xapk

# Decompile with Fernflower
bash plugins/android-reverse-engineering/skills/android-reverse-engineering/scripts/decompile.sh --engine fernflower library.jar

# Run both engines and compare
bash plugins/android-reverse-engineering/skills/android-reverse-engineering/scripts/decompile.sh --engine both --deobf app.apk

# Find API calls
bash plugins/android-reverse-engineering/skills/android-reverse-engineering/scripts/find-api-calls.sh output/sources/
bash plugins/android-reverse-engineering/skills/android-reverse-engineering/scripts/find-api-calls.sh output/sources/ --retrofit
bash plugins/android-reverse-engineering/skills/android-reverse-engineering/scripts/find-api-calls.sh output/sources/ --urls

```

Repository Structure

```

android-reverse-engineering-skill/
├── .claude-plugin/
│   └── marketplace.json          # Marketplace catalog
├── plugins/
│   └── android-reverse-engineering/
│       ├── .claude-plugin/
│       │   └── plugin.json      # Plugin manifest
│       └── skills/

```

```

└─ android-reverse-engineering/
    └─ SKILL.md # Core workflow (5 phases)
    └─ references/
        └─ setup-guide.md
        └─ jadx-usage.md
        └─ fernflower-usage.md
        └─ api-extraction-patterns.md
        └─ call-flow-analysis.md
    └─ scripts/
        └─ check-deps.sh # Bash
        └─ check-deps.ps1 # PowerShell
        └─ install-dep.sh
        └─ install-dep.ps1
        └─ decompile.sh
        └─ decompile.ps1
        └─ find-api-calls.sh
        └─ find-api-calls.ps1
└─ commands/
    └─ decompile.md # /decompile slash command
└─ LICENSE
└─ README.md

```

References

- [jadx – Dex to Java decompiler](#)
- [Fernflower – JetBrains analytical decompiler](#)
- [Vineflower – Fernflower community fork](#)
- [dex2jar – DEX to JAR converter](#)
- [apktool – Android resource decoder](#)

Acknowledgments

Thanks to the contributors who have shaped this skill:

- [@philjn](#) – Native Windows / PowerShell support (check-deps.ps1, install-dep.ps1, decompile.ps1, find-api-calls.ps1) and split/bundled APK detection in decompile.sh (#8)
- [@txhno](#) – Migration to the maintained [ThexXTURBOXx/dex2jar](#) fork (#12)
- [@muqiao215](#) – Decompile partial-success handling, Fernflower timeout safeguard, intermediate-artifact directory (#10)
- [@kevinaimonster](#) – Chinese localization (SKILL.md discovery keywords) (#4)

Disclaimer

This plugin is provided strictly for **lawful purposes**, including but not limited to:

- Security research and authorized penetration testing
- Interoperability analysis permitted under applicable law (e.g., EU Directive 2009/24/EC, US DMCA §1201(f))
- Malware analysis and incident response

- Educational use and CTF competitions

You are solely responsible for ensuring that your use of this tool complies with all applicable laws, regulations, and terms of service. Unauthorized reverse engineering of software you do not own or do not have permission to analyze may violate intellectual property laws and computer fraud statutes in your jurisdiction.

The authors disclaim any liability for misuse of this tool.

License

Apache 2.0 — see [LICENSE](#)